Two Notes and a Tempo: A Minimal Procedural Music Generator Driven by Game Difficulty

Samar Khajuria, T00714740
COMP 4980, Category 3: Computationally Generated Music

***Abstract*—Procedural content generation (PCG) lets a game build its own content, so that content can vary between plays. Background music is a good target for this. This paper presents a very small generator, written in Java with no outside libraries, that makes the music for each level out of only two notes. The melody mostly swaps between a low C and a high C. On each beat there is a fixed chance that it repeats the note instead of swapping, and that is what makes every level different. A drum track marks each beat. The only input from the game is the difficulty level, which sets the speed. Level one plays at 80 beats per minute and level five plays at 200, so a thirty-two-note piece lasts 24.0 s on the easiest level and 9.6 s on the hardest. The generator writes a playable MIDI file for each level in a few milliseconds, so a game only needs to store the level number instead of an audio file.**

***Index Terms*—procedural content generation, game music, adaptive audio, MIDI, Java**

I. Introduction

Procedural content generation builds game content with algorithms instead of by hand, so that each play can be different [1]. Music is a good fit for this. A normal soundtrack plays the same way every time a level is replayed, but generated music can change on every run and can react to what the player is doing [2]. Using computers to compose is not new. Hiller and Isaacson made the Illiac Suite from rules and random choices in the 1950s [3], and later systems use Markov models, grammars and other algorithms [4].

Most systems that generate music try to make it rich and varied. This project asks the opposite question. What is the smallest generator that still makes useful game music and still reacts to the game? The answer here uses two notes, one chance value and one input from the game. Keeping it this small makes every design choice easy to see, and it makes the link between the game and the music easy to show.

II. Method

A. The Two-Note Melody

The melody uses only two notes. One is MIDI note 60, which is middle C. The other is MIDI note 72, which is the C one octave higher. Each piece is thirty-two notes long and plays one note per beat. The generator keeps track of the current note, writes it down, and then picks the next one. Normally it swaps to the other note, which gives a steady pattern of low, high, low, high. The two notes are an octave apart, so they have the same note name and always sound right together. This means no key or scale has to be defined.

B. Randomness

A generator that only swaps would make the same piece every time. To stop that, the generator rolls a whole number from 0 to 99 on each beat. If that number is under 25, it keeps the current note instead of swapping. A repeat shifts the pattern onto the opposite foot, so a run such as low, high, low, low, high appears and the steady pulse breaks in a different place on every level. The random number generator starts from the level number, so each level always gets the same music back, while different levels get different music.

This is a two-state Markov process. The next note depends only on the current one, with a 0.75 chance of changing and a 0.25 chance of staying [5]. The repeat chance is the one setting that controls how regular the music sounds. A value of zero gives a perfectly even pattern. Larger values break up the pulse more often.

A drum track plays a bass drum on every beat on MIDI channel 10. The drum is the same on every level, so it sets a steady pulse. The melody repeats are then heard as variation against that pulse rather than as noise.

C. Difficulty and Tempo

The generator takes one input from the game, which is the level number. Level one plays at a base speed of 80 beats per minute. Every level above that adds a fixed step of 30, so level five plays at 200. Every piece has thirty-two notes, so raising the speed makes the piece shorter and raises the note rate by the same amount. The note patterns vary with the level seed, but the pitches, repeat chance and drum stay fixed. In the taxonomy of Shaker et al. [1] this generator is stochastic, constructive and automatic. It is also adaptive rather than generic, because its output follows the state of the game and not just a random seed.

III. Implementation

The generator is a single Java class of about 220 lines. MIDI files are written with javax.sound.midi and the figure is drawn with java.awt. Both of these come with Java, so the program builds and runs with no outside libraries. The arithmetic uses whole numbers wherever possible, including the speed calculation and the repeat decision, so the code reads without type conversions. Each level is written as a MIDI file with two tracks, one for the melody and one for the drum, and it opens in programs that support MIDI. A full run writes five MIDI files and one figure in under a second.

IV. Results

TABLE I
Music Settings and Output for Each Level

Level	Tempo (bpm)	Length (s)	Notes/s	Repeats
1	80	24.0	1.33	7
2	110	17.5	1.83	7
3	140	13.7	2.33	6
4	170	11.3	2.83	8
5	200	9.6	3.33	14

Table I shows what the difficulty input does. The speed rises evenly from 80 to 200 beats per minute. That shortens a thirty-two-note piece from 24.0 s to 9.6 s and raises the note rate from 1.33 to 3.33 notes per second, which is a factor of 2.5. The number of repeated notes ranges from 6 to 14, with a mean of 8.4. A repeat chance of 0.25 over thirty-one decisions gives an expected value of 7.75. With only thirty-one decisions per level, that spread is normal randomness and not an effect of difficulty. The repeat chance stays constant. Each level sets the tempo and seeds a repeatable note pattern. Fig. 1 shows the patterns.

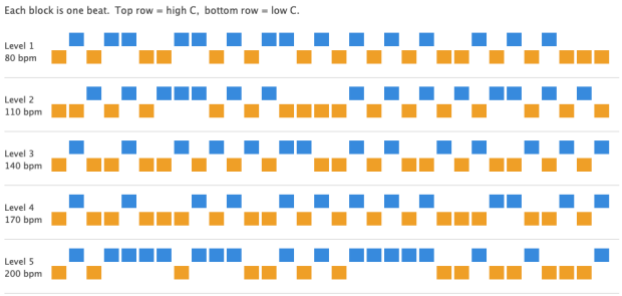


Fig. 1. The pattern for each level. Each block is one beat. Blocks on the upper row are the high C and blocks on the lower row are the low C. The swapping shows up as a zig-zag, and the random repeats show up as two blocks next to each other on the same row.

V. Conclusion

A generator built from two notes, one chance value and one input makes music that is different on every level and that clearly tracks difficulty. The cost is low. It is one source file with no outside libraries, it takes a few milliseconds per piece, and a game stores a level number instead of an audio file. The limits come straight from the design. With only two notes there is no real melody, and the result is closer to an adaptive pulse than to a tune. Nothing was tested with listeners, so the idea that faster music feels harder rests on convention and not on evidence. Good next steps would be to add a small set of notes so the generator can suggest a key, to change the drum as well as the speed, and to drive the difficulty input from live game state instead of the level number, in the way the adaptive pacing works in Left 4 Dead [2].

References

[1] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Cham, Switzerland: Springer, 2016.

[2] C. Plut and P. Pasquier, “Generative music in video games: State of the art, challenges, and prospects,” *Entertainment Computing*, vol. 33, Art. no. 100337, 2020, doi: 10.1016/j.entcom.2019.100337.

[3] L. A. Hiller and L. M. Isaacson, *Experimental Music: Composition with an Electronic Computer*. New York, NY, USA: McGraw-Hill, 1959.

[4] G. Nierhaus, *Algorithmic Composition: Paradigms of Automated Music Generation*. Vienna, Austria: Springer, 2009.

[5] C. Ames, “The Markov process as a compositional model: A survey and tutorial,” *Leonardo*, vol. 22, no. 2, pp. 175–187, 1989.